Ninaivu

நினைவு · memory

For whoever looks after the technical side.

Technical administrator guide · version 4.2.0

Docker
on a NAS

Source
a checkout

Command line
flags and tools

Contents

Everything in this guide, and the page it is on.

Technical administrator guide 3

This page is for whoever looks after the technical side of Ninaivu: the source code, Docker, running from a checkout, the command line,...

Where things are 3
More on the portable downloads 4
Docker 5
From a checkout 6
Running it from the command line 7
Reaching Ninaivu from outside 9
The TV album 10
Tuning to the machine 11
A second pair of face models (a trial) 16
Content fingerprints (a trial) 16
The change journal (a trial) 17
Ask Ninaivu (a trial) 18
Our Year (a trial) 18
Moments inside videos (a trial) 19
Then and now (a trial) 19
A life in photographs (a trial) 20
Share without strangers (a trial) 21
Find this print (a trial) 21
Photo stacks (a trial) 22
Young faces, older names (a trial) 23
Festival albums (a trial) 23
Letters read aloud (a trial) 24
Talk about this photo (a trial) 25
Updating to a new version 26
Moving the index to another folder 29
Environment variables and the state folder 30
Further reading 30

NINAIVU 4.2.0

Technical administrator guide

This page is for whoever looks after the technical side of Ninaivu: the source code, Docker, running from a checkout, the command line, environment variables and where to report a problem. The guide tells a household how to set up Ninaivu with the download for their computer and use it. This page is in English only.

Where things are

Source code	https://github.com/javajaga-usa/Ninaivu
Downloads (the page the guide points to)	https://github.com/javajaga-usa/Ninaivu/releases/latest
Reporting a problem	https://github.com/javajaga-usa/Ninaivu/issues
Extensions (Creative Studio, Gemini and the rest)	https://github.com/javajaga-usa/Ninaivu/tree/main/extensions
The docs site	https://javajaga-usa.github.io/Ninaivu/

When reporting a problem, say which version (**System → Home & extensions** shows it under *This home*), what was done, and what the log said.

More on the portable downloads

Ninaivu ships as one portable folder per computer — Windows x64, macOS on Apple silicon and Linux x64 — each with its own Python and every package, and nothing installed. What each folder keeps, and where:

	INDEX, SETTINGS, ACCOUNTS	AI MODELS	LOGS	CONTROL PANEL'S OWN FILES
Windows	`data`	`data\ai-models`	`logs`	`data\control\.ninaivu-control`
macOS	`data`	`data/ai-models`	`logs`	`data/control/.ninaivu-control`
Linux	`data`	`data/ai-models`	`logs`	`data/control/.ninaivu-control`

The program is apart from all of it: `app` on Windows and Linux, `Ninaivu.app` on a Mac. Up to 1.11, Windows and Linux kept the data inside `app` (`app\data`, `app\ai-models`, `app\.ninaivu-control`); a 1.12 folder that finds it there moves it to `data` on its first start (`ninaivu/desktop/portable.py`).

The launchers (`Ninaivu.exe`, `Ninaivu.app`, `./ninaivu`) set `NINAIVU_HOME`, `NINAIVU_STATE_DIR`, `NINAIVU_AI_MODELS_DIR`, `NINAIVU_LOG_DIR`, `NINAIVU_PORTABLE` and `NINAIVU_PORTABLE_ROOT` (the folder itself) to those folders, and on Windows and Linux `NINAIVU_PROGRAM_DIR` to `app`; nothing outside the folder is written, except *Start Ninaivu when I sign in* (a registry value on Windows, a LaunchAgent on a Mac) when someone turns it on.

- **macOS.** The app keeps its data in the folder only when it sits beside the hidden `.ninaivu-portable` file the zip has. Dragged on its own into Applications, it keeps to `~/.ninaivu` and `~/Library/Application Support/Ninaivu`, as an installed app would. `open -a Ninaivu --args --tray` opens it minimised to its menu-bar icon.
- **Linux.** One download, `linux-x64`, for a 64-bit PC running Debian 12 or 13, Ubuntu or another current distribution; every release is opened on Debian 12 and 13 before it is published. There is no arm64 (Raspberry Pi) download. `./ninaivu --server [PHOTOS] [options]` runs the server alone in the terminal, with the same folders; `./ninaivu --tray` opens Ninaivu minimised to its tray icon; `./ninaivu --where` prints the folders. `ninaivu` is the folder's one starter, a small program that runs `app/ninaivu.sh`, and may be reached through a symbolic link (`ln -s ~/Ninaivu-…/ninaivu ~/bin/ninaivu`).

The Linux system service

The portable folder does not set itself up as a service. To have systemd start it at boot, as its own user rather than root, put the folder somewhere that user can write (`/opt/ninaivu`), and adapt `installers/systemd/ninaivu.service`:

```
User=ninaivu
ExecStart=/opt/ninaivu/ninaivu --server /var/lib/ninaivu/photos --port 5000
ReadWritePaths=/opt/ninaivu /var/lib/ninaivu
```

```
sudo useradd --system --user-group --home-dir /opt/ninaivu --shell /usr/sbin/nologin
ninaivu
sudo tar -xzf Ninaivu-<version>-linux-x64-portable.tar.gz -C /opt
sudo mv /opt/Ninaivu-<version>-linux-x64-portable /opt/ninaivu
sudo chown -R ninaivu: /opt/ninaivu
sudo cp installers/systemd/ninaivu.service /etc/systemd/system/   # edited as above
sudo systemctl enable --now ninaivu
```

A library elsewhere has to be readable and writable by that user, for example `sudo setfacl -R -m u:ninaivu:rwX -m d:u:ninaivu:rwX /srv/photos`, and named in `ReadWritePaths=`. Run the maintenance commands as that user too, so the files they write stay its own: `sudo -u ninaivu /opt/ninaivu/ninaivu --server reset-password`.

`sudo systemctl stop ninaivu` lets Ninaivu finish what is in hand: the service allows 45 seconds (`TimeoutStopSec`), and the log names anything that had not stopped by then.

Docker

For a NAS or a Linux box:

```
git clone https://github.com/javajaga-usa/Ninaivu.git
cd Ninaivu
cp installers/docker/.env.example installers/docker/.env   # set MEDIA_DIR to your
photo folder
docker compose -f installers/docker/docker-compose.yml up -d
```

The family app answers on port 5000 to the whole network; the console on 3000, published on the NAS itself only (`127.0.0.1:3000`). To make the administrator from another computer, open the console through an SSH tunnel or on the NAS, and give the **setup code** printed in `docker logs ninaivu`. The `.env` file is only for folders and ports: everything about how Ninaivu behaves is chosen in the console, and nothing in `.env` undoes it at the next restart. The image leaves out

search by description unless built with `--build-arg WITH_AI=1`. Compose builds the image from the checkout; each release also publishes one, `ghcr.io/javajaga-usa/ninaivu:<version>` (and `:latest`), for x64 and arm64 alike, to use in `docker-compose.yml` instead of building. `docker stop` and `docker compose down` give it 45 seconds to finish what is in hand (`stop_grace_period`), rather than Docker's ten. The operations guide covers systemd, reverse proxies, HTTPS and large libraries.

From a checkout

Running from the source needs Python 3.14 or newer (the portable downloads bring their own).

```
git clone https://github.com/javajaga-usa/Ninaivu.git
cd Ninaivu
python -m pip install -e . -r requirements/requirements-desktop.txt
python -m ninaivu ~/Pictures         # the server
python -m ninaivu.desktop.app        # the Control Panel, with its tray icon
```

Or let a starter in `launcher/` set it up:

```
launcher\start.cmd ~/Pictures        # Windows: or double-click it
sh launcher/start.sh ~/Pictures      # macOS and Linux
```

The launcher makes a virtual environment, installs what is missing (nothing large: search by description is added later, if you want it), serves HTTPS on port 443 — or 8443 on Linux, where an ordinary user may not use 443, the same port every time — finds free ports and opens your browser. `python -m ninaivu <folder>` runs the server directly, on plain HTTP and port 80 (8080 without the right to use 80), unless given `--https`.

On Windows, `launcher\start.cmd` needs no Python beforehand: when it finds no Python 3.14 or newer, it installs Python 3.14 for the current user first (with winget, or from python.org when winget is missing), then carries on. No administrator rights are needed. **launcher\Start - Ninaivu Control Panel.vbs** opens the Control Panel once `launcher\start.cmd` has run.

On a Mac, `sh launcher/start.sh` installs Python 3.14 if the Mac has no Python 3.14 or newer (after asking: with Homebrew when the Mac has it, otherwise the installer from python.org, which is run only when it carries the Python Software Foundation's signature, and for which macOS asks your password). `launcher/Ninaivu.command` does the same from the Finder and asks once which folder holds your photos. More on the Control Panel.

A checkout keeps its log in `.ninaivu-control/server.log` beside it.

Videos for guests and share links need ffmpeg. A phone writes the place a video was filmed into the file, and guests, share links and family members whose copies leave out the home location are given a copy without it, made by ffmpeg. Without ffmpeg on the computer Ninaivu runs on, those people are told the video cannot be shown rather than sent the original. The Docker image includes it; elsewhere, install ffmpeg from the system's packages (on Linux) or Homebrew (on a Mac).

Running it from the command line

`python -m ninaivu <folder>` (from a checkout) starts the server, and these are the flags worth knowing:

FLAG	
`--admin USER:PASSWORD`	make the administrator without a browser
`--port`, `--admin-port`	the two ports (80 — 443 with HTTPS — and 3000 by default)
`--local-only`	this computer only; phones cannot reach it
`--https`, `--cert`, `--key`	HTTPS with Ninaivu's own certificate, or yours. `--https` only decides the first start; after that the console's **HTTPS** switch (Server page) does. `--http` forces plain HTTP
`--ai auto|clip|light|off`	the image model; *auto* follows the hardware tier
`--faces`, `--ocr`, `--places`	turn the passes on from the start
`--no-watch`	do not watch the folders for changes
`--open-browsing` / `--private`	whether visitors may look without signing in
`--lock-roots`	confine the folder picker to the library folders
`--admin-host 0.0.0.0`	open the console to the network from the start
`--strict-port`	refuse to start rather than move to another port

FLAG	
`--name`, `--no-mdns`	the name on the home network (`ninaivu.local`), or none
`--map-tiles`	OpenStreetMap tiles on the map (they show roughly where photographs were taken)

When the port is taken by another program, Ninaivu moves to the next free one and says so; `--port` chooses another, `--strict-port` refuses to move.

And the maintenance commands, which work in a portable folder too (on Linux, `./ninaivu --server <command>`):

COMMAND	
`ninaivu backup --out <folder>`	a copy of the index, settings and certificate
`ninaivu list-backups <folder>`	the copies in a folder
`ninaivu restore <file>`	put one back (Ninaivu stopped)
`ninaivu reroot --from <old> --to <new>`	tell the index the library moved
`ninaivu reset-password <name>`	a new password for someone, from this computer (a forgotten administrator password); the Control Panel's **Reset a password…** does the same
`ninaivu import-lite <file>`	bring a Ninaivu Lite household across: people, who sees what, favourites, albums, share links (Ninaivu stopped, after its first scan)
`ninaivu offsite-restore --recovery <file> <copy> <output>`	put the photographs back from an off-site copy, on any computer
`ninaivu space`	where the library's bytes are and what a light library would save (estimates; reads only, safe while Ninaivu runs; `--json` for the raw figures)
`ninaivu fit <folder>`	before copying the library to another disk: the kind of disk the folder is on, and every file whose name, case, Unicode spelling or size it could not keep, with a name that would work (reads only)

The storage check reads the whole library and takes hours on a large one. Until the console has a button for it, `POST /api/admin/scrubber/stop` (signed in to the console) stops it after the file it is reading; a restart does not start it again, and Start carries on from where it stopped.

Reaching Ninaivu from outside

Ninaivu decides for every request whether it came from home or from the internet. Home is the house's own network (IPv6 included), this computer, Tailscale and WireGuard. Any public address counts as the internet, and so do a forwarded router port, a tunnel or proxy Ninaivu was not told about (forwarded headers from a peer not in `trusted_proxies` and `NINAIVU_TRUSTED_PROXY_ADDRESSES`), and Tailscale Funnel. From the internet:

- profiles set to *tap to enter* do not open, and nobody can browse without signing in;
- plain HTTP straight from a public address is refused, on every path;
- the console refuses every request, whatever `--admin-host` or *Open this console from other devices* say, unless `console_from_internet` (**Advanced settings → Remote access**, off by default) is turned on for a tunnel or proxy on purpose;
- 20 wrong administrator passwords in half an hour pause sign-in by username for 30 minutes, then an hour, then two; the computer Ninaivu runs on can still sign in.

Everywhere, uploads are refused while the server's disk would be left with under 2 GB free (phone backups already were), an upload that is really a playlist (`#EXTM3U`, an ffconcat list) is refused, and signing out over HTTPS empties the browser's cache of thumbnails.

What the household should set up, once:

1. Reach Ninaivu from away through **Tailscale** (or WireGuard). Forward no port on the router, switch UPnP off there, and keep inbound IPv6 blocked (most routers' default). Never turn on Tailscale Funnel for Ninaivu.
2. Turn on **HTTPS certificates** in Tailscale's admin page, so Ninaivu serves its `.ts.net` name with a certificate phones accept (step by step).
3. Turn on the computer's **firewall**, allowing Ninaivu, and leave *Open this console from other devices* off unless it is needed.
4. A **long administrator password** (14 characters or more, or four random words), kept in a password manager and used nowhere else.
5. A **PIN on every profile**, and open browsing off if anyone not fully trusted uses the Wi-Fi.
6. **Disk encryption** on the computer (FileVault, BitLocker or LUKS): Ninaivu does not encrypt the library or the index on this computer; only the cloud and off-site copies are encrypted.
7. **Two-step sign-in** on the accounts around Ninaivu: Tailscale, Google Drive, the off-site storage service, GitHub.
8. **Keep things up to date**: the operating system, Tailscale, ffmpeg, and Ninaivu when a release comes out (it never updates itself).
9. On a fork of the repository, GitHub's **Dependabot security updates**.

Ninaivu itself has no two-step sign-in yet, and HTTPS is not on by default on the home network; reach it by its Tailscale name at home too, or turn on the **HTTPS** switch on the Server page, if anyone untrusted may be on the Wi-Fi.

The TV album

The TV album puts one album on the televisions at home, with no app to install. Ninaivu answers as a DLNA media server, which is what a smart TV's own media player (often called *Media*, *Media Player* or the TV's source list), VLC (*Local network → Universal Plug'n'Play*) and Kodi (*Videos* or *Pictures → Add source → UPnP devices*) look for. The TV shows it as "*home name* TV album", and the remote walks through the photographs and videos, newest first.

It is off until an administrator chooses the album, in the console under **System → Server → TV album**: pick the album (or *Off*), change the port if 8200 is taken, and press **Save**. The block then says whether it is showing, how many photographs and videos the TV sees, and lists the addresses a TV can be given by hand when it does not find the server by itself. Behind it are the settings `tv_album` (the album's number; 0 is off) and `tv_port`, and `GET` / `PUT /api/admin/tv-album` for scripts.

What the TV sees is kept narrow, because DLNA has no sign-in and anything on the home network can ask:

- only the chosen album, and of it only the photographs and videos a family member could see: never anything hidden, flagged, in the bin, outside the family's date limit, or a sound recording, even when the album holds one;
- only on the home network: a device with a public address, or one reaching Ninaivu over Tailscale or WireGuard, is refused, and nothing is answered with *Network access* off;
- every address starts with a random secret, so a guessed address finds nothing. **New addresses** in the same block makes a new one, and every address handed out before stops working; the TVs find the new one by themselves the next time they look for media servers.

Photographs are sent as the same upright copy, at most 2560 pixels and with no location or camera details in it, that a share link sends. Videos are sent as they are. The firewall must let the TV reach `tv_port` (TCP) and UDP port 1900, which is where TVs look for media servers; if 1900 cannot be had, the TV album still runs and the TV can be given its address by hand.

Tuning to the machine

At every start Ninaivu measures the computer (its cores, memory, graphics processor, whether it is a Raspberry Pi or another single-board computer, and whether the library is on a spinning disk) and picks a profile: **Small box** (a Pi, under 6 GB of memory, or two cores), **Everyday computer**, or **Powerful computer** (eight cores and 16 GB, or a graphics processor and 16 GB). The profile sizes the indexing workers, the analysis threads, the web threads, the image model's batch, the moments described per video, the backup uploads at once and the index cache. **Peak performance** is only ever chosen by hand (or by the desktop panel's Performance mode): every core (18 of 18) and up to 95% of the memory, the rest of the memory left for the operating system. The desktop panel's Standard mode is **Everyday computer**, half the cores (9 of 18). **Powerful computer** uses 80% of the cores and keeps at least two back (14 of 18). Every core the system reports is counted, of every kind: an Apple chip's Super, Performance and Efficiency cores alike. Where the orientation survey runs beside the scan (a graphics processor and a solid-state library), its readers are part of the same share: Peak keeps two cores for it. A profile chosen on the Tuning page wins over the panel's mode, so set it back to automatic there for Performance mode to take effect.

The console's **System → Tuning** page shows what was measured, the profile, what the numbers are expected to use, and every number with where it came from. Any of them can be set outright and put back to automatic; most apply at once, the indexing workers at the next scan, and the web threads and the index cache after a restart, which the page offers. A number given at start (`--workers`, `NINAIVU_SERVER_THREADS`, `NINAIVU_COMPUTE_THREADS`, or written into `config.json` by hand) is kept unless it is set on the page. The choices are saved in `config.json` as `tuning_profile` and `tuning`.

The job engine

Since 3.0 every scan works on the job engine; the 2.x `job_engine` switch is gone, and a `config.json` that still says it is read as before with the word left out. A scan indexes new and changed files, reads text and finds faces in worker processes, one per scan worker (the Tuning page's number), held to what half the free memory allows at about 350 MB each. Text and faces are tasks in `engine.db` beside the index: they run side by side, carry on after a stop or a restart, and a file that crashes a worker is tried alone and set aside after two crashes instead of stopping Ninaivu. Answers are written to the index in groups of up to 64. Pictures for sound files, the description of each video from its moments (read on the workers, described by the image model in the server) and content fingerprints (two files at a time) are tasks in the same list, and grouping faces into people is worked out on a worker process of its own once there are 2,000 faces. A run of fewer than 32 tasks (the photograph or two the watcher found) works on threads of the server instead, on the same list and in the same way, since starting processes would take longer than the work; faces, and videos on a computer without ffmpeg, always use processes. Describing photographs, place names, imports, the storage check, the second copy and backups run as before. `engine.db`

holds nothing that cannot be worked out again: deleting it while Ninaivu is stopped only forgets which files were set aside. `tools/bench_engine.py` compares a scan on threads and on worker processes, on the computer it runs on.

Maintenance mode

Activity → Sharing this computer → Maintenance gives the whole computer to the library's own work for a while, announced to the household as downtime. While it is on:

- every job runs at once and flat out: indexing, the analysis (tags, faces, words, descriptions, voice stories, video moments, fingerprints), the straightening survey, the cloud upload and the storage check. Nothing makes way for a video or for somebody browsing, and the daily catch-up's stretch and its wait for an idle computer are lifted. The analysis a library still owes is started straight away, whether or not the daily catch-up is on.
- the Tuning profile is **Peak**, every core (on the Mac mini, 12 cores and up to 95% of its 48 GB). A scan that was already running keeps the number of workers it started with; the next one takes the new number.
- family members and guests see a **Down for maintenance** notice in the family app instead of albums, and share links answer the same. The notice goes by itself, and the gallery comes back, within half a minute of the end. Administrators keep the console and the family app.

Nothing is changed or lost. **End maintenance** (or choosing another mode) puts back the mode and the Tuning profile that were there before. Switches that are off stay off: maintenance does not describe photographs while **Describe photos** is off.

The daily catch-up (a trial in 2.0)

`daily_catchup` on the console's **Advanced settings** page (in `config.json`, or `NINAIVU_DAILY_CATCHUP=1` for one launch) keeps the heavy analysis for one stretch of the day: the image model's look at what is in each photograph, finding faces, reading words in photographs, describing photographs and videos, writing down voice stories, fingerprints and the straightening survey. Place names and grouping faces into people run as before. The stretch begins at `daily_catchup_at` (02:00) and lasts `daily_catchup_hours` (6; 24 means all day). Inside it the work runs flat out only while the computer is idle: nobody has used the family app for five minutes, no video is playing, and the other programs on the computer have used under 15% of the processor for three minutes (Ninaivu's own processes are not counted). It stops when somebody opens the app or other programs use more than 35% for half a minute, and carries on when it is quiet again. When the stretch opens, Ninaivu starts the analysis a library still owes (without walking it again) and then the straightening survey; a scan or survey somebody stopped is left until the next day. Indexing new photographs, the cloud upload and the storage check still follow the **Background work** choice. **Activity → Sharing this computer** says why the analysis is waiting.

The computer must be awake when the stretch begins. Ninaivu cannot wake a sleeping computer and does not change its power settings. Once Ninaivu is running it keeps the system awake (the screen may still sleep). On a Mac that is put to sleep or shut down at night, schedule macOS to wake or start a few minutes before the stretch, in Terminal (System Settings has no schedule since macOS 13):

```
sudo pmset repeat wakeorpoweron MTWRFSU 01:55:00
pmset -g sched          # shows the schedule
sudo pmset repeat cancel  # removes it
```

and tick **Start Ninaivu when I sign in** in the Control Panel, with automatic log-in on if the Mac is shut down rather than asleep. A Mac mini or Mac Studio on mains power wakes reliably this way; a laptop wakes for it only on mains power with the lid open.

The on-device AI engine (a trial of 2.0)

`ai_runtime`, on by itself since 3.1 once one of its models is downloaded (`false` in `config.json`, or `NINAIVU_AI_RUNTIME=0` for one launch, turns it off), switches on the engine Ninaivu 2.0 is built around: one process of its own, the *model host*, that runs the newer models on this computer and nowhere else. Today it writes down what is said in voice stories. It needs two things, both fetched only when asked for:

1. the speech reader, **Voice stories as words** under Extras on the **Home & extensions** page (sherpa-onnx, about 45 MB; or `pip install -r requirements/requirements-speech.txt`);
2. a speech model on the **AI models** page: **Whisper large-v3** (1.07 GB download, about 1.8 GB on disk and 2.5 GB of memory while it works), which writes down Tamil and English and translates a Tamil story into English, or **Whisper turbo** (564 MB, about 1 GB on disk), several times quicker with no translation. large-v3 is used whenever it is there.

With both in place, every story is written down in the background: shortly after it is told, and any already told within a minute of the switch going on. A story's words are searched with the words typed beside it, and the story's API gives them, in its language and in English, with the start and end of each piece for subtitles. The household's languages are `speech_languages` (AI settings), most used first: `ta,en` unless changed. A story heard as any other language is taken to be the first one. It gives way to the household like any analysis (Workload), and a story that cannot be written down twice is left until a newer model.

The model host lowers its own priority, leaves after ten minutes with nothing to do (its memory is the computer's again), and a crash in it costs one story an attempt, not the gallery. Off, nothing of it runs. `tools/try_speech.py recording.m4a` writes down a recording with the downloaded model and prints what it heard and how long it took, so a household can try it on its own voices before relying on it.

Photographs: search without PyTorch, and a description of each

The engine also runs the image models, through ONNX Runtime instead of PyTorch. They need the image engine, **Search and descriptions without PyTorch** under Extras (ONNX Runtime and a tokenizer, about 25 MB; or `pip install -r requirements/requirements-image-engine.txt`), and on the **AI models** page:

- **AI search without PyTorch** (SigLIP 2 B/16, about 1.5 GB): finding photos by what is in them, tags and the explicit-content check, from the engine instead of PyTorch. It is the same model as **AI search (SigLIP 2)** in the ONNX export Immich publishes, prepared and tokenised as PyTorch's is, so its vectors are PyTorch's within rounding and share their model id: changing between the two does not index the library again. Search moves to the engine at the next start. (A library indexed with SigLIP 2 before 2.0 is indexed once more, by either engine: until then PyTorch prepared SigLIP 2's photographs as it prepares CLIP's, which matched what the model was trained on less well.) Its process is a second model host, so a search never waits behind a long story; it leaves after half an hour without a search.
- **A description of every photo** (Florence-2 base, about 550 MB; or Florence-2 large at full precision, about 3.3 GB on disk and 4 GB of memory while it works, which the AI models page suggests on a computer with 24 GB of memory or more and which is used whenever it is downloaded): a sentence or two about what is in each photograph and video, written in the background from the gallery's 640-pixel preview, newest photographs first. Descriptions are searched with each item's other words (and rank first among the search model's answers when they hold every word typed), and `GET /api/asset/<id>` gives one as `description` for the viewer. They are the engine's, kept apart from the caption: a caption somebody wrote is never replaced, an item hidden from AI keeps none, and the copy of the index sent to Drive carries none. A picture the model cannot describe is tried once more, then left until a newer model. Progress is on the activity strip and in `GET /api/admin/ai-models` under `engine`.

`tools/check_onnx_search.py <photos>` compares the engine's vectors with PyTorch's on the household's own photographs and times both, and `tools/try_captions.py <photos>` describes a few and says how long the whole library would take on this computer.

Each model stays in memory while its work lasts, which for the descriptions of a large library is days: the writing model about 9 GB, speech large-v3 about 2.5 GB, Florence-2 large about 4 GB and the search model about 1.5 GB, some 17 GB together with Ninaivu's own workers beside them. A 48 GB computer holds them all; on 16 GB, download one at a time and let each finish its work before the next.

Memory films (a trial of 2.0)

`memory_films`, on by itself since 3.1, with or without the writing model (`false` in `config.json`, or `NINAIVU_MEMORY_FILMS=0` for one launch, turns it off), puts **Make a film** beside **Make a book** in the family app. Ninaivu picks the best photographs and clips of an album, a trip or a person, moves gently towards the faces, tells the voice stories over their photographs, cuts on the beat of the

music and writes an MP4 under `films/` in Ninaivu's own folder. A film belongs to whoever made it; it is checked again before it is played, and refused once a moment in it can no longer be seen by them. Like an iPhone memory, a film plays soft music unless somebody chooses otherwise: one of four pieces of Ninaivu's own (Morning light, Courtyard veena, Together, Festival day), the one for the film's pace unless another is picked, with **Listen** to hear it first. The music is written on this computer as the film is made (`features/films/music.py`), to the film's length and with the cuts on its beat, so nothing is downloaded and there is no song licence to think about; it fades in and out and gives way whenever a voice story or a clip's own sound plays. **No music** leaves it out. Songs in the library are offered to administrators; anybody may bring a song of their own (up to 40 MB, kept under `films/music/`). Subtitles need the voice stories written down (`ai_runtime` above). One film is made at a time, in up to eight parts side by side at low priority; ffmpeg must be installed (Extras). Deleting a profile deletes its films and songs.

The Make a film screen can also **draft the chapters**: from dates and places, or written by the on-device writing model (**Writing in Tamil and English (Gemma 3 12B)** on the AI models page, 7.1 GB, with **Writing in Tamil and English** under Extras to run it, and `ai_runtime` on). Its answer is checked before it is shown, and every word can be changed. `tools/try_writing.py "a question"` tries the model on this computer. For a person, the screen offers two more kinds of film: **growing up** (the face held by the eyes through the years, with the year or their age) and **then and now** (pairs years apart, side by side). Both use the faces found on the People page, so they need the person's faces confirmed or matched, with dates. With `photo_frame` on too, the frame's **Play our films** plays the profile's films.

Speaking to search (a trial)

`voice_search`, on by itself since 3.1 once a speech model is downloaded (`false` in `config.json`, or `NINAIVU_VOICE_SEARCH=0` for one launch, turns it off), puts a microphone in the family app's search box for family members and administrators. What is said, in Tamil or English, is written down by the voice-story model (**Voice stories as words** on the AI models page, large-v3 or turbo, with **Voice stories as words** under Extras to run it) and searched as if typed; a Tamil search is searched in its English translation when large-v3 is downloaded. The recording is removed as soon as it is read. Browsers allow the microphone only over the secure (https) address. It does not need `ai_runtime`.

Spoken searches and Add colour (below) use a second model host, the **quick host**, so they never wait behind a long story or the descriptions; it uses the whole of the tuning's threads, holds about 2.5 GB with large-v3 and 1 GB with the colouring model, and lets them go after five idle minutes.

Colour for black-and-white photographs (a trial)

`colour_photos`, on by itself since 3.1 once the colouring model is downloaded (`false` in `config.json`, or `NINAIVU_COLOUR_PHOTOS=0` for one launch, turns it off), offers **Add colour** in the viewer's menu on a photograph with no colour of its own (grey or a single tint, such as sepia). It needs **Colour for black-and-white photographs (DDColor)** from the AI models page (980 MB) and

ONNX Runtime under Extras. The colour is shown over the original to compare, and **Keep the colour copy** saves a copy beside the original as an edit is saved (a family member's waits for approval); the original is never changed. A colour copy offers **Before and after** at any time. Up to two photographs are coloured at once; one takes a few seconds.

A second pair of face models (a trial)

Faces are found and told apart by OpenCV's YuNet and SFace unless `face_model` on the console's **Advanced settings** page (under AI; `NINAIVU_FACE_MODEL`) is `arcface`: InsightFace's SCRFD detector and ArcFace recogniser, downloaded on the **AI models** page as **Finding and grouping faces, more accurate (ArcFace)** (289 MB from the project's GitHub release, of which the two models, 191 MB, are kept) and run through ONNX Runtime (the local AI tools' package, `requirements/requirements-ai-local.txt`). They are better at small and turned faces, at relatives, and at a child's face over the years. Their licence allows non-commercial use only; the page says so.

Switching has the next scan look at every photograph again: the two recognisers' values are not comparable. The faces already found stay on the People page until each photograph's turn comes, a confirmed name is kept by the face's position (one name to one face, the closest), and grouping compares only faces found by the same recogniser. "This is not that person" answers and each person's cover face go with the face too. A name the new search cannot put on any face is not dropped: it waits and is tried again by the next search, and `GET /api/faces/not-carried` lists the ones still waiting. `tools/try_faces.py <photos>` runs both pairs over a folder of the household's own photographs and prints what each found and how long it took, to try before switching.

Content fingerprints (a trial)

`content_fingerprints`, on by itself since 3.1 (`false` in `config.json`, or `NINAIVU_CONTENT_FINGERPRINTS=0` for one launch, turns it off), gives every file in the library a fingerprint of its bytes, BLAKE3, kept in the index beside the file's size and modified time. Nothing on disk changes: the year/month/day folders stay as they are, and the fingerprints are only an index over them.

- **Moving or renaming keeps everything.** A photograph dragged to another day's folder in Finder or Explorer, or a folder renamed, used to be indexed as a new file, and the faces, story, albums, favourites and edits of the old one were dropped with it. With fingerprints, a scan that finds a file gone and a new file with the same bytes keeps the row under the new name. A date that came from the folder or the file's name follows the move; one from the camera does not change. Only a file fingerprinted before it moved can be followed, and only within one library folder.

- **Exact copies across every drive.** `GET /api/admin/fingerprints` reports how many files have a fingerprint in each library folder and lists the files that are byte-for-byte copies of each other, most space first.
- **What it costs.** Every file is read once, as the last pass of a scan, after the place names, text and faces: about the time a storage check takes. A stop or a restart carries on where it was. After that, only new and edited files are read. Without the `blake3` package (it is in every download) the fingerprints are BLAKE2b and are made again once it is there.

The change journal (a trial)

`change_journal`, on by itself since 3.1 (`false` in `config.json`, or `NINAIVU_CHANGE_JOURNAL=0` for one launch, turns it off), has the index write down every change to its list of files as it happens: added, changed (its bytes), renamed, moved, put in the bin, taken out of it, and gone, with when. The rows are written by triggers on the index, so every way a file changes is recorded alike, and they are kept for a year. With `content_fingerprints` on as well, a renamed or moved file is recorded as such rather than as one gone and one new, and a file that is gone keeps its fingerprint in the record.

- **Health → Library changes** shows the last 30 days, a line for each day, kind and folder, and warns when hundreds of files went or were written over in one folder within minutes. That is what a folder replaced by an older copy, or a sync that ran the wrong way, looks like; the warning only says so, and the mirror and the backups still have the files.
- **For tools and other parts of Ninaivu:** `GET /api/admin/journal` (the summary), `GET /api/admin/journal/changes?after=N&limit=500&kind=removed` (a cursor: pass back `next`; `gap` says rows were trimmed since) and `GET /api/admin/journal/item/<id>` (one item's history).
- **On a Mac, a quicker start.** macOS keeps a record of which folders changed on each drive (FSEvents). After Ninaivu has read a whole library folder it remembers where that record stood; at the next start it asks the drive what changed since and reads only those folders, or none. It walks the whole library as before whenever the answer is not certain: the drive dropped events or keeps no record (exFAT, network shares), a different drive is mounted under the same name, more than 64 folders changed, or the whole folder was last read more than 7 days ago. Windows and Linux walk as before.

Ask Ninaivu (a trial)

`ask_ninaivu`, on by itself since 3.1 once the writing model is downloaded (`false` in `config.json`, or `NINAIVU_ASK=0`, turns it off), puts **Ask Ninaivu** in the family app's top bar for family members and administrators. A question in Tamil or English is read like a search ("Paati", "Ooty", "2019", "last year", "போன வருடம்"), the photographs it is about are found inside the asker's own limits, and the answer says how many, the first or most recent with where, the years, the places and who else is in them, with the photographs.

With the on-device AI engine on (`ai_runtime`) and **Writing in Tamil and English** downloaded on the AI models page, the writing model also reads the question (better for Tamil, held to names and places the asker can see) and writes the answer in a few sentences in the question's language, from those facts only. It sees nothing of a photograph only administrators may see. The model host loads the model on the first question and lets it go after ten idle minutes; how long an answer takes on the Mac has not been measured yet (`tools/try_writing.py` times the model on its own). Without the model, the answer is the same facts as plain sentences. Nothing is sent anywhere.

Our Year (a trial)

`our_year`, on by itself since 3.1 (`false` in `config.json`, or `NINAIVU_OUR_YEAR=0` for one launch, turns it off), puts **Our year** in the family app's **Create** menu for everybody who may open the gallery. It shows a year as a set of cards, in Tamil or English: the counts and the busiest month and day, who is in the most photographs and who appears for the first time, who is always together, the places and the furthest from home (set the home zone for that one), the trips, the family's favourite photograph, the voice stories, and the best photograph of every month, ending with a book or a film of the year. A year needs at least twenty photographs to be offered.

It is counting over the index, nothing more: no pixel is read and nothing is sent anywhere, and every count goes through the viewer's own limits, so a hidden photograph or a folder somebody is kept out of never shows. A guest sees the counts, months and photographs but no names, places or stories. With the on-device AI engine on (`ai_runtime`) and **Writing in Tamil and English** downloaded, the writing model also writes a few opening sentences from the year's counts and names; they are kept in memory until the library changes or Ninaivu restarts. How long that takes on the Mac has not been measured yet.

Moments inside videos (a trial)

`video_moments`, off since October 2026 (`"video_moments": true` in `config.json`, or `NINAIVU_VIDEO_MOMENTS=1` for one launch, turns it on; it is no longer on the Advanced page), lets a search find the second inside a long video. Above a search's results a strip, **Moments in videos**, shows the scenes of videos that match the words, each with its time ("candles" finds 14:32 of the twenty-minute birthday video); a tap opens the video in the viewer at that second. Everybody who may search sees it, inside their own limits, as with the picture search.

It needs ffmpeg and the picture search (the image model, or the on-device engine's search with `ai_runtime`). In the background, once per video, every video of 20 seconds or more is read at a frame every two seconds (a longer video more sparsely, at most 400 frames, from its keyframes where it has them), each frame is turned into a search vector by the same model the gallery uses, and frames in a row that look alike are folded into one scene that keeps the second it starts at. It is analysis, so it waits for the household like the other passes, and it shows on the console's activity strip as **Finding moments in videos**. A video changed on disk, or a different search model, is read again; a video hidden from AI keeps no moments. Nothing is sent anywhere. How long a long video takes on the Mac, and whether the scenes are cut in the right places on the family's own videos, has not been measured yet.

Then and now (a trial)

`then_and_now`, on by itself since 3.1 (`false` in `config.json`, or `NINAIVU_THEN_AND_NOW=0` for one launch, turns it off), puts **Then and now** in the family app's **Create** menu. It pairs photographs years apart, at least five years, under three headings:

- **Places**: two photographs from the same small square of the map (about 170 metres) that also look like the same view to the picture search, such as the temple tank in 1994 and in 2021. An old print scanned without a place is paired with a newer photograph only when the two look like the very same view. Without the picture search, two photographs within 60 metres are paired by the map alone.
- **People**: each named person's early photographs beside their late ones (the 24 people in the most photographs), their best faces.
- **At the same age**: a parent beside their child at the same age, for the parents and children on the family tree. The family types the year each was born once, on that heading; it is kept in the index and used only for this.

A pair opens large with the newer photograph over the older up to a handle dragged across (or the arrow keys), or the two side by side. A face is held by the eyes in both, so the girl of 1966 and the boy of 1996 sit in the same place. With `memory_films` on, ticked pairs become a memory film, side

by side. A strip above the gallery, **Then and now, this week**, shows the pairs whose older photograph was taken this week, years ago.

Everything goes through the viewer's own limits; screenshots are left out, and a guest sees the places without their names and no people. It compares search vectors and faces the library already has, so nothing new is read or downloaded and nothing is sent anywhere. The pairs are worked out when the page is first opened, then kept until faces or search vectors change, or for an hour. How long that first look takes on the real library, and whether the "looks like the same view" thresholds (in `ninaivu/features/then_and_now/media.py`) suit the family's photographs, has not been measured yet.

A life in photographs (a trial)

`life_story`, on by itself since 3.1 (`false` in `config.json`, or `NINAIVU_LIFE_STORY=0` for one launch, turns it off), puts **A life in photographs** in the family app's **Create** menu, and a **Life story** button beside **Make a book** when the gallery shows one person. It is for family members and administrators; a guest, who is never told who is in a photograph, does not get it.

Choose somebody (anyone named in at least twelve dated photographs) and their life is told decade by decade. Their best photograph opens the page with their name, the years and a few opening words. Each decade is a chapter with a title and two or three sentences, the best photographs of those years (no two from the same day), who **joined their photographs** then (a husband, a daughter, a grandchild, with how they are related on the family tree), who was most often beside them, where they were and their trips, and the voice stories told about those photographs, their own first, to play where they are. A decade with fewer than four photographs joins the next one. A rail of the decades follows the reading down the page. The year somebody was born is asked once, on their page (the same year Then and now uses), and each chapter then says how old they were. The page ends with **Make a book** and **Make a film** of that person.

The facts are counted on this computer from what the library already knows: dates, places, trips, named faces, the family tree and the stories. When the on-device writing model is downloaded (AI models page, with `ai_runtime` on), it writes each chapter's title and sentences in Tamil or English from those facts only, while the page shows its own plain sentences. What it writes is kept in the index for exactly the facts it was written from, so the next visit opens with it, and someone who may see fewer photographs is never shown words written from photographs they cannot open. Without the model the plain sentences are the story. Nothing is sent anywhere. How long Gemma 3 12B takes to write a long life on the Mac has not been measured yet; the page waits for up to five minutes.

Share without strangers (a trial)

`share_blur_strangers` on the console's **Advanced settings** page (or `NINAIVU_SHARE_BLUR_STRANGERS=1`) blurs, on every share link, the faces nobody in the household has named: the photograph, its browser copy and the small tiles of a shared album. A face counts as named when it is given to a person on the People page, by somebody saying yes or by the matcher being sure; a suggestion nobody answered is still a stranger. The family app's share box says so before the link is made.

The face pass keeps only faces good enough to recognise somebody by, and the small, soft and half-turned faces at the back are exactly the strangers, so each shared photograph is looked at again with the face model at the lower bar the retouching tools use, and every face found that is not a named one is blurred: reduced to a few blocks, smoothed and feathered into the photograph, past recognising. The blurred copy is made the first time the link shows the photograph and kept with the other browser copies, named for the faces it was made from, so naming somebody later makes the next visit show them clear.

A photograph that cannot be looked at goes out blurred all over rather than as it is: when the face model is not downloaded and the face pass has never been over it, and always for a hidden photograph an administrator shares on purpose, because AI does not read hidden items. Videos are shared as they are; their faces are not known frame by frame. The original never leaves while the switch is on. Everything is done on this computer; nothing is sent anywhere. How long the first look takes for a large shared album on the Mac, and whether the lower bar finds the faces at the back of the family's own group photographs, has not been measured yet.

Find this print (a trial)

`find_print`, on by itself since 3.1 (`false` in `config.json`, or `NINAIVU_FIND_PRINT=0` for one launch, turns it off), puts **Find this print** in the family app's **Add photos** menu, beside **Scan old prints**. It is for family members and administrators; guests, who cannot scan prints, do not get it.

Somebody photographs one framed print on a wall, or one print in an album, and Ninaivu says which photograph in the library it was printed from. The print is found in the phone's picture by the same code that scans prints, flattened and stood up, then compared two ways:

- **The picture fingerprint** every photograph already keeps (the one the duplicate finder uses, `perceptual_hash`). It needs no model and is kept for hidden photographs too. The print is also tried with its edges cut away and with the picture inside its mount, in every quarter turn, because a frame or a mount left in the crop changes the fingerprint.
- **The picture search** (SigLIP 2 or CLIP vectors), which forgives glare, fading and a crooked crop better, while its model is loaded and only for photographs it has read (never hidden ones).

The best matches are shown, those that are surely the same photograph marked **The same photo**, others as **Might be this**. When nothing is certainly it, **Take it in** hands the same photograph to **Scan old prints**, with the print already found, to be saved as any scan is (an administrator's straight into the library, a family member's into **Review → Uploads**).

Every match is narrowed to what the person may open, the same check the gallery uses: a family member is never told that a hidden or explicit photograph, a folder they are kept out of or someone else's "Only to me" item matched. An administrator sees every match, with hidden and explicit ones marked. The phone's picture is compared on this computer and sent nowhere; it is held, as a scan is, for six hours in case it is taken in, then cleared. What counts as the same photograph (`SURE_BITS`, `MAYBE_BITS`, `SURE_LOOK` and `MAYBE_LOOK` in `ninaivu/features/find_print/media.py`) is a guess that has not been tried on the family's real prints yet.

Photo stacks (a trial)

`photo_stacks`, on by itself since 3.1 (`false` in `config.json`, or `NINAIVU_PHOTO_STACKS=0` for one launch, turns it off), shows a burst, and a photograph with the copies edited from it, as one tile in the family app's timeline: the sharpest frame, with the count in its corner. The viewer shows every frame in a bar above the filmstrip. Family members and administrators can **Put on top** another frame or **Show separately** (and **Stack again**); guests only look. Each choice is kept by the stack and outlives every rescan.

A stack is the near-identical photographs the scan already groups for the duplicates page (`perceptual_hash`, `duplicate_distance`) that were taken on the same day, sit in the same folder, and are seen alike by everybody: the same visibility, the same explicit flag and the same "Only to me" owner. So a hidden or explicit frame is never stacked under a family photograph, and nobody can reach a frame through a stack that they could not open anyway; the frames are read through the same check the gallery uses. Exact copies of a file in the same folder stack too; copies in other folders do not.

Only the plain timeline, a day, a folder, an occasion and a kind of item are collapsed. Search, people, albums, places, favourites, a camera, the quality filters and the duplicates page still list every frame, because a frame under the top one may match where the top one does not. Library counts are not changed. Nothing is moved or deleted.

The marks are worked out again on this computer whenever the library has changed, at most every 15 seconds, when the gallery is opened; hiding, flagging, binning or moving one frame shows every frame of its stack again at once, until then. The first start after the update that brought stacks rebuilds five of the gallery's indexes once, switched on or not (a few seconds on a large library). Not yet tried on the Mac's library.

Young faces, older names (a trial)

`young_faces` on the console's **Advanced settings** page (or `NINAIVU_YOUNG_FACES=1`) puts **Young faces, older names** in the family app's **Create** menu, for family members and administrators (a guest is never told who is in a photograph). It suggests who an unnamed face in an older photograph may be, from the faces the family has named in later years.

The ordinary matcher compares a face with a person's *average* named face, which for Paati is a woman of fifty, so a girl of eight in a 1962 print is never offered. Here each person is held to their **youngest** named faces instead: the confirmed faces from the first eight years of their dated named photographs (a person needs at least three dated named faces). A face from at least two years before their earliest named photograph is compared with those, and the further back it is, the more it may differ: it starts out needing 0.10 more than the recogniser's own same-person line, and each year back takes 0.005 off, so twenty years back needs only the line itself. Nothing below that line is ever offered, here or anywhere. A photograph whose year cannot be believed (no date, or only the date the file was copied or scanned) gets 0.04. The year somebody was born (the same year Then and now and A life in photographs keep, asked for on this page too) leaves out photographs from before it and gives each face an age. Nobody is suggested for a photograph they are already named in, or after somebody said no. A parent, grandparent, brother, sister, husband or wife named in the same photograph (from the family tree) earns another 0.03 and is shown as the reason. When a second person fits within the sibling margin, both are offered. These numbers are guesses (`ninaivu/media/young_faces.py`) and have not been tried on the family's own photographs.

Yes confirms the face through the same call as the console's review queue and the "Who is this?" answers, so the person's likeness is rebuilt from it. A family member may name only a face nobody has named, on a photograph they can open, as somebody in their own People row: the same widening the "Who is this?" review makes. **No** is kept as "not this person". **Ask a relative** makes an ordinary "Who is this?" link for the face, which shows the face and never the suggested name. Only photographs the viewer may open are read (hidden ones and other people's "Only to me" items never for a family member; an administrator sees everything, explicit photographs included), a person is described only by faces in photographs the viewer may open, and screenshots and documents are left out. The suggestions are worked out when the page is opened, reading the unnamed faces in pieces of 5,000, and kept until faces change or for an hour. How long that first look takes on the real library has not been measured yet. Nothing is sent anywhere.

Festival albums (a trial)

`festival_albums`, on by itself since 3.1 (`false` in `config.json`, or `NINAIVU_FESTIVAL_ALBUMS=0` for one launch, turns it off), puts **Festival albums** in the family app's **Create** menu. Every Pongal, Tamil New Year, Krishna Jayanthi, Vinayagar Chathurthi, Navaratri, Deepavali, Karthigai Deepam

and Christmas the library holds becomes one album that grows each year, found without anybody tagging a photograph.

Each festival's day is worked out on this computer from the Tamil calendar, from where the sun and the moon were that year (Pongal on the first of Thai, Deepavali on the waning fourteenth before Aippasi's new moon, Karthigai Deepam on the evening the moon stands in Krittika, Navaratri from its first day to Vijayadashami), so there is no table to keep up to date. The album takes the photographs of the few days around each day (Bhogi to Kaanum Pongal, the eve of Deepavali to the day after) and the picture search checks them: a day is kept when one of its photographs looks more like the festival (lamps, sparklers, kolam, golu dolls on their steps, sugarcane and the Pongal pot, a Ganesha idol, the Christmas tree) than like an ordinary photograph, and then the whole day joins, the family photograph in new clothes as much as the lamps. A day where nothing looks like the festival (an office outing that fell on Deepavali) is left out. Photographs the picture search has not read yet join on their dates, and without the picture search a year is found from its dates alone (at least four photographs on them), marked **by date**. A date the camera did not record (the file's own times, which can be the day it was copied) never counts, and screenshots and documents are left out.

A festival opens on its latest photograph and its years side by side, the oldest on the left: each year its day and its best photographs, so the doorstep lamps of 1994 sit beside last year's. A row of years jumps along. A tap opens that year's photographs in the viewer, and **Make a book** keeps every year in one book. Each card also says when the festival next falls.

Everybody who may open the gallery may open it, guests too, through their own limits; nobody is named. Family members never see hidden photographs or other people's "Only to me" items in it; administrators see everything the gallery shows them, explicit photographs included. Nothing is sent anywhere. The albums are worked out when first opened, then kept until the library or its search vectors change, or for an hour. How long that first look takes on the real library, and whether the picture search's festival words suit the family's photographs, has not been measured yet; the words are in `ninaivu/features/festivals/media.py`. A festival on the edge of a rule (a lunar day that begins at dawn) can come out a day from a family's calendar, which the days around it cover.

Letters read aloud (a trial)

`letters_aloud` in `config.json` (or `NINAIVU_LETTERS_ALOUD=1`; since 3.1 it is no longer on the Advanced settings page) adds **Read aloud** to the viewer's menu on every photograph, for family members and administrators (not guests). It opens the photograph beside its words: an old letter, a recipe card, a note in a grandparent's hand.

The words. Read the words from the photo asks the text reader already in the app (**Reading the words in photographs** under Extras; the `ocr_enabled` pass need not be on) to read the photograph, turned upright and at 2,600 pixels on its long side. It reads printed English well; Tamil, and handwriting of any kind, it seldom reads well, so what it finds is offered in a box to check against the photograph and correct before **Keep the words**. The words can also be typed from the start. They are kept in the index with the language they are read in (Tamil when most of the letters are Tamil script), whether they were read or typed, and who kept them last.

The voices. Two downloads on the **AI models** page, Meta's MMS text-to-speech models (CC-BY-NC 4.0, non-commercial use, which a household's own library is): **Letters read aloud in English** (about 115 MB, the sherpa-onnx project's ONNX export, run by sherpa-onnx, the package voice stories use) and **Letters read aloud in Tamil** (about 150 MB, as Meta publishes it, run by transformers on PyTorch: **Letters read aloud in Tamil** under Extras installs both, about 2 GB). The voices are the models' own; there is no way to make them sound like anybody in the family. They run in the quick model host, like colour for black-and-white photographs, whether or not `ai_runtime` is on, and need nothing else of it.

The letter is read sentence by sentence, a little slower than the models' own pace, with a short pause after each sentence and a longer one between paragraphs. Numbers are said as words (years as "nineteen sixty-two" in English; digit by digit in Tamil). An English word in a Tamil letter is read by the English voice when it is downloaded, and left out when it is not. The reading is a WAV in `letters/` in the state folder, kept so that playing the letter again is instant; changing the words removes it, and the next **Read aloud** makes a new one. The sentence being read lights up on the page, and pressing a sentence plays from there.

In memory films. A photograph with a kept reading and no voice story is narrated by its reading in a memory film (`memory_films`), with its sentences as subtitles, as if the letter had been told as a story.

How long the Tamil voice takes for a page on the Mac has not been measured yet (in the cloud, the English voice read a four-sentence letter in about a second on four cores); the page waits for up to seven minutes.

Talk about this photo (a trial)

`photo_talk`, on by itself since 3.1 once the writing model and its vision part are downloaded (`false` in `config.json`, or `NINAIVU_PHOTO_TALK=0` for one launch, turns it off), puts **Talk about this photo** in the viewer's menu for family members and administrators. It opens a side panel beside the photograph: type a question, or tap one of the suggestions (**What is written here?**, **Where might this be?**, **What is happening here?**, **Tell me about this photo**), and the writing model looks at the photograph itself and answers in a few sentences, in Tamil when the question is in Tamil and in English otherwise. "What does the banner say?" is read off the banner, Tamil script

and all; "which temple is this?" is answered from what the tower looks like, or the answer says it cannot tell. A follow-up question carries the talk so far. A guest, who is never told who is in a photograph, does not get it.

It needs `ai_runtime` on, the writing model (**Writing in Tamil and English (Gemma 3 12B)**, 7.1 GB) and its vision part (**Seeing photographs (Gemma 3 12B's vision part)**, about 850 MB) from the AI models page, and **Writing in Tamil and English** under Extras to run them. The two are one model in memory, shared with Ask Ninaivu, the films' chapters and A life in photographs; the vision part adds about 1 GB while it is loaded.

The photograph is turned upright and handed to the model host as a JPEG of at most 1,024 pixels a side, readable only by Ninaivu, and removed as soon as the answer is back. Beside it the model is told only what the asker already sees on that photograph in the viewer: when it was taken (not when a date comes from the file), the town, the names the family gave the people in it, the engine's description and the family's own caption; it is told not to give anybody another name. Neither the question nor the answer is kept, on the computer or in the log; the panel keeps the talk while the viewer is open. **A hidden photograph (hidden by hand, by its folder, or kept "Only to me") and one marked explicit are never shown to the model**, not even by an administrator who can see them: the viewer does not offer it there, and the route refuses. Two questions are answered at once, and a question waits up to fifteen seconds for the model host to finish something else before it is told Ninaivu is busy. How long Gemma 3 12B takes to look and answer on the Mac has not been measured yet; the first question after a while also loads the model.

Updating to a new version

An update replaces the program and nothing else. The photographs are never read or written by an update, and neither is the state folder (the index, `config.json`, the thumbnails, `setup-code.txt`, the keys) or the AI models.

Ninaivu never looks on the internet for a newer version, and nothing is ever downloaded or installed by Ninaivu itself. An update is a newer portable folder someone downloads from the releases page, when they choose to.

A portable folder updates itself from the one beside it. Unpack the newer download beside the folder in use, on the same drive, and open it. Its Control Panel finds the older folder (the most recently used portable Ninaivu of the same system in the same parent folder, whose library no newer Ninaivu has opened) and offers to bring the library across. On yes it asks the older server to stop through its run file, as Stop does, then moves the older folder's data, models and Control Panel files into the new `data` folder with renames, so nothing is copied and a library of any size moves in a moment. A folder on another drive is refused rather than copied. The older folder keeps

its program, a `LIBRARY-MOVED.txt` for people and a `.ninaivu-moved.json` that stops it from being offered again; the newer one records where its library came from in `data/brought-from.json`. If the older copy was set to start at sign-in, the sign-in start is moved to the newer one.

The newer version's first start copies the index and settings before it changes them (`data/backups/before-update`, below). **Go back to …** in the newer Control Panel, shown while the older folder is still there, stops Ninaivu, puts the index and settings back from that copy, and moves everything back to the older folder. What was done in the newer version since (albums, names, people, capsules, settings) is not carried back, but a copy of the library as the newer version left it is taken first, in `data/backups/before-go-back`, and can be restored like any backup. A go-back cut short (a file held open) is finished by pressing **Go back** again.

If the newer version cannot start at all on this computer (it stops before it is serving, twice), the Control Panel goes back **by itself**: it is the same Go back, taken only while the newer version has never once been serving on this library, so nothing done in it is lost, and the older Ninaivu is opened. Each start that does reach serving is noted in `data/version.json` (`good_at`); after that, going back is always the person's choice. From a terminal, with the launcher's environment: `python -m ninaivu.desktop.portable --status`, `--bring FOLDER` and `--go-back` do the same.

For the Linux service, `systemctl stop ninaivu` before updating; for Docker, the `pull` and `up -d` below do it.

HOW IT WAS INSTALLED	HOW TO UPDATE	WHAT IS REPLACED	WHAT STAYS AS IT WAS
Windows	extract the newer zip beside the old folder, open its `Ninaivu.exe` and bring the library across	everything in the zip (`Ninaivu.exe`, `app\python`, `app\tools`)	`data` (index, settings, people, models, the Control Panel's settings), the library
macOS	unzip the newer one beside the old folder, open its `Ninaivu.app` and bring the library across	`Ninaivu.app`	`data` (index, settings, people, models, the Control Panel's settings), the library
Linux	unpack the newer `.tar.gz` beside the old folder, run its `./ninaivu` and bring the library across	`ninaivu`, `app/python`	`data`, the library

HOW IT WAS INSTALLED	HOW TO UPDATE	WHAT IS REPLACED	WHAT STAYS AS IT WAS
An installed Ninaivu from 1.10 or earlier	**Mac:** drag the new `Ninaivu.app` on its own over the old one in Applications: on its own it keeps to `~/.ninaivu` as before. **Windows, Linux:** the installed copy keeps working; to move to the portable folder, stop it and copy its state folder (`%USERPROFILE%\.ninaivu` , `~/.ninaivu` or `~/.local/state/ninaivu`) into the new folder as `data`	the program	the old state folder, until it is removed by hand
Docker	`docker compose pull` (or `build`), then `up -d`	the image	the `ninaivu_state` volume and `MEDIA_DIR`
A checkout	`git pull` , then start it again	the source	the state folder, `.ai-models` , the library

In Docker, never add `-v` to `docker compose down` : that deletes the state volume, which is the index. The library itself is still untouched.

The first start of a new version is the only time anything of the old setup changes: it adds what it needs to `index.db` and `archive.db` , in place. Before that, it copies the whole state folder into `<state>/backups/before-update` (the same verified bundle the scheduled backup makes, the three most recent kept) and notes the version in `<state>/version.json` . If the copy cannot be made (a full disk) Ninaivu does not start, and says so; `NINAIVU_SKIP_UPDATE_BACKUP=1` starts it once without the copy, for someone who has a backup of their own.

Going back to an older version. An older Ninaivu refuses to start on an index a newer one has changed in a way it does not know, and names the copy to restore; it never writes over it. To go back: install the older version, then `ninaivu list-backups "<state>/backups/before-update"` and `ninaivu restore <bundle>` (with Ninaivu stopped), which puts back the state from just before the update. Anything done after the update (new albums, names) is not in that copy; the photographs are not affected either way. Versions up to 1.0.4 do not have this check, so going back to one of those needs the restore first. Going back from 4.0 to any 3.x always takes this restore: 3.x does not open an index 4.0 has opened. Every 3.x index still opens in 4.0 and later, which bring it up to date themselves.

Moving the index to another folder

The index, thumbnails and settings (the state folder) can move to another folder or drive from the console: **Migration → Move the index to another folder**. Your photographs stay where they are.

- Give a new folder, an empty one, or one that already holds a copy of this index (a copy made by hand, an earlier attempt, or the folder the index moved from). With a copy, only what is missing or different is copied.
- Before Ninaivu restarts, the folder is checked: that Ninaivu can write in it, that the drive has room for what is left to copy, and that the index holds no symbolic links.
- Ninaivu restarts and does the move before anything else runs, so its own work (describing, scanning, consolidating, backups) waits for it and resumes after. On Linux it also waits, up to five minutes, for any other program still writing in the index folder.
- Every file is read back from the new folder, from the drive rather than from memory on Linux, and compared with the original. A file that differs is copied again; files changed during the copy are copied again too.
- Only when everything matches does the old folder get its forwarding note, `index-location.json`, which sends every launcher to the new folder. Keep that file. The old folder stays as a recovery copy, which the console can clean up once you are happy.
- The new folder's databases are then opened and their tables listed (every page was already compared during the copy), and repaired if need be (see *Checking and repairing the index* in the backup guide).

If the move cannot finish, the index stays where it was, the console says why and where it tried, and what was copied is kept: moving to the same folder again continues from it. The Control Panel shows the progress while it runs, and the server log has every step (`Index move: …`). Choose a drive with a Linux or macOS file system for the index (ext4, APFS); a Windows (NTFS) drive read through Linux is not a good home for it.

Reading the index from another folder (Advanced settings)

On a one-computer Ninaivu and on the Primary, the `state_dir` row of **Advanced settings** points Ninaivu at the index already in another folder: the one used before a new download, say, when the new copy started with an empty index of its own.

1. Type the folder that holds the index (the folder with `index.db` in it) and press **Use this folder…**.
2. Ninaivu shows what is in it: how many photographs, its libraries, and which version last opened it.
3. Enter your password and press **Restart and use it**.

Nothing is copied or moved. At the restart the folder Ninaivu used until then gets a forwarding note, `index-location.json`, to the new one, and the index in it is kept exactly as it was: deleting that note goes back to it. The index read from now on is checked, and repaired if need be, as the start goes on.

Ninaivu refuses a folder with no index in it (moving this index somewhere is **Move the index**, above), the folder it uses now, an index another Ninaivu is using, one a newer Ninaivu last opened, and one that has itself moved elsewhere. The setting is never written into `config.json`, and is not offered on the Secondary or on a read-only copy that follows another computer (the family view served with `--replica-of`).

Environment variables and the state folder

Some settings also read an environment variable (`NINAIVU_STATE_DIR`, `NINAIVU_AI_ENGINE`, `NINAIVU_HARDWARE_TIER`, `NINAIVU_FACES`, `NINAIVU_AI_RUNTIME`, …; the full list is in `ninaivu/server/config.py`). Folders and ports from the environment always apply; anything about behaviour only fills in what nobody chose in the console. A number out of range, from the environment or typed into `config.json` by hand, is brought within its limits (one from the environment is named in the log); a value of the wrong kind in `config.json` is set aside with a message and Ninaivu starts anyway. The state directory (`~/.ninaivu`, or `$XDG_DATA_HOME/ninaivu`, or `NINAIVU_STATE_DIR`) holds the index, the thumbnails and `config.json`. The AI models live in `.ai-models` in a checkout; an installed copy keeps them in a folder of the user's own, which an upgrade does not replace (`%LOCALAPPDATA%\Ninaivu\ai-models` on Windows, `~/Library/Application Support/Ninaivu/ai-models` on a Mac, `~/.local/state/ninaivu/ai-models` on Linux). `NINAIVU_AI_MODELS_DIR` or the `ai_models_dir` setting moves them; the Docker image sets the former to `/data/state/ai-models`, on the state volume.

Further reading

- Operations and production — Docker, systemd, reverse proxies, large libraries, backups.
- The tray, the Control Panel and HTTPS
- Backup and recovery and encryption and the recovery file
- Moving to another machine
- Contributing and security



Ninaivu

For whoever looks after the technical side.

THE GUIDE, ONLINE
javajaga-usa.github.io/Ninaivu/

SOURCE
github.com/javajaga-usa/Ninaivu

REPORT A PROBLEM
github.com/javajaga-usa/Ninaivu/issues

Ninaivu 4.2.0 · MIT licence